

Equipo 2:

Capitán:

- Leonel Fachinelli.

Integrantes:

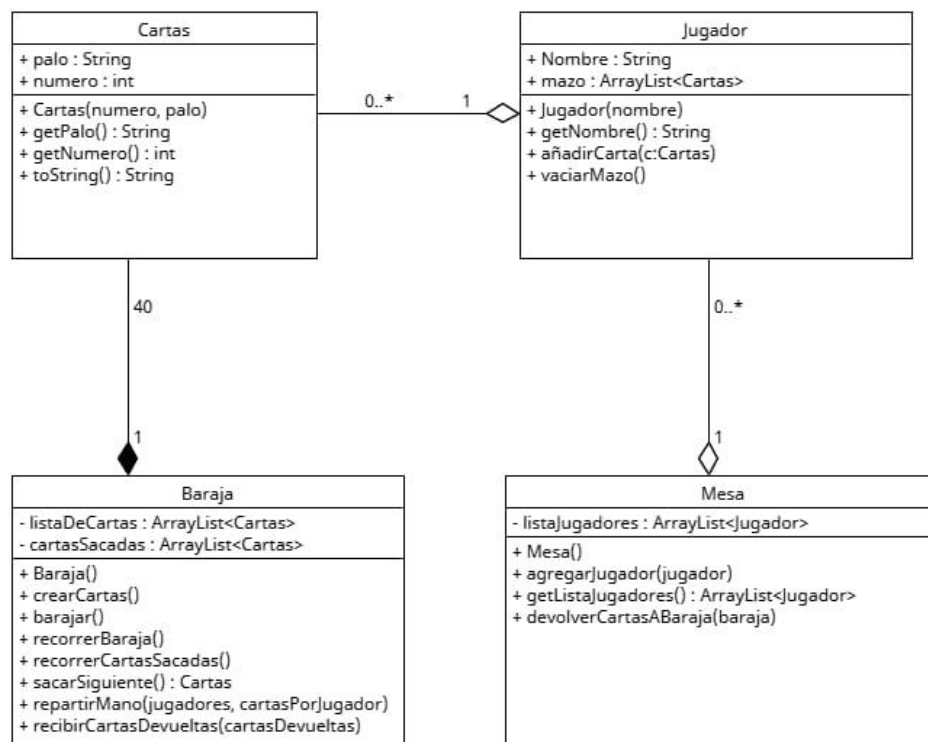
- Maximiliano Reinoso, Xavier Pappalardo y Lautaro Gómez.

JUEGO DE BARAJAS ESPAÑOLAS

ANÁLISIS DE PROBLEMA:

El problema a resolver fue el crear un sistema de barajas españolas, el cual cumpliera los requerimientos y funcionalidades típicas de este tipo de juegos, tales como barajar las cartas, repartirlas a cada jugador, crear una mesa con una cierta cantidad de jugadores, etc., y que cumpliera con los criterios de integridad necesarios, como quitar las cartas ya repartidas del mazo para que no las puedan sacar de nuevo si ya están repartidas.

DIAGRAMA UML:



DECISIONES DE DISEÑO:

Para resolver el problema, decidimos crear un total de 4 clases además del Main, las cuáles fueron Baraja, Mesa, Cartas y Jugador.

Esto lo decidimos para que el usuario posea la libertad de crear una o varias mesas de juego con sus propios jugadores, que no tienen que ser necesariamente la misma cantidad entre una mesa y otra, y una baraja independiente entre cada mesa.

En cada mesa, los jugadores tendrían a su vez su respectiva mano con las cartas repartidas, y una baraja que pueda realizar las acciones propias de una baraja clásica, la cual estaría compuesta por cartas con sus palos y números correspondientes.

A continuación, definiremos los atributos y métodos creados para cada clase correspondiente y su funcionalidad dentro del diseño.

CLASES CREADAS:

- **Baraja:**

Atributos:

- listaDeCartas: private ArrayList que contiene los objetos ‘Cartas’ correspondientes a las cartas que quedan en la baraja.
- cartasSacadas: private ArrayList que contiene los objetos ‘Cartas’ correspondientes a las cartas ya sacadas de la baraja.

Métodos:

- crearCartas: public void que emplea bucles ‘for i’ para recorrer la lista ‘palos’, que contiene los símbolos de las cartas a crear, y luego otro para crear una nueva carta y añadirla a la lista, con una condición para impedir que se creen cartas con los números 8 y 9.
- barajar:
- recorrerBaraja: public void que emplea un bucle ‘for’ que recorre la lista ‘listaDeCartas’.

- recorrerCartasSacadas: public void con una condición que verifica que se hayan sacado cartas, para, en caso de que se cumpla, recorrer la lista 'cartasSacadas' con un bucle 'for'.
 - sacarSiguiente: public Cartas con una condición que verifica que queden cartas en el mazo, y en caso afirmativo, quita la primer carta que quede en él (La que posea el índice 0).
 - repartirMano: public void que emplea un bucle 'for' para recorrer la lista de jugadores en la mesa 'jugadoresEnMesa', para luego, mediante un bucle 'for i' repartir a cada jugador las cartas correspondientes mediante el método sacarSiguiente, con una condición que verifica que hayan cartas en la baraja, por lo que si el resultado de sacarSiguiente es 'null' (No hay más cartas en la baraja) detiene automáticamente el bucle con su respectivo 'return'.
 - recibirCartasDevueltas: public void que devuelve todas las cartas devueltas a la lista 'listaDeCartas' que contiene las cartas de la baraja, y vacía la lista 'cartasSacadas' que contenía las cartas repartidas.
- **Cartas:**
 - Atributos:
 - palo: public String que contiene el palo de la carta (Espada, Basto, Oro, Copa).
 - numero: public int con el número de la carta.
 - Métodos:
 - toString: public String que devuelve un mensaje donde se especifican el palo y número de la carta, los cuales se devuelven dentro de él como cadenas de caracteres.
- **Jugador:**
 - Atributos:
 - Nombre: public String con el nombre del jugador.
 - mazo: public ArrayList que contiene una lista de objetos 'Cartas' con las cartas repartidas a dicho jugador.
 - Métodos:

- añadirCartas: public void que recibe objetos 'Cartas' y los añade al mazo del jugador.
- vaciarMazo: public void que vacía el mazo del jugador.
- **Mesa:**
 - Atributos:
 - listaJugadores: private ArrayList que contiene objetos 'Jugador' correspondientes a los jugadores en la mesa.
 - Métodos:
 - agregarJugador: public void que recibe objetos 'Jugador' y los añade a la lista 'listaJugadores'.
 - devolverCartasABaraja: public void que recibe objetos 'Baraja' y mediante un bucle 'for' recorre la lista 'listaJugadores' para así recibir las cartas de cada jugador y vaciar su mazo respectivo.
- **Main:** Esta clase no posee atributos y solo posee su método public static void 'main' respectivo, donde se ejecuta el código del programa.

FUNCIONALIDADES:

- **Mesa:**
 - Crear una lista de jugadores.
 - Añadir un jugador a la mesa.
 - Retornar la lista de jugadores actual.
 - Recoger las cartas de los jugadores y devolverlas a la baraja.
- **Jugador:**
 - Asignarle un nombre.
 - Crear un mazo con la lista de cartas que posee.
 - Retornar su nombre.
 - Añadir una carta a su mazo.
 - Vaciar su mazo.
- **Baraja:**
 - Crear una lista con las cartas que quedan en la baraja.

- Crear una lista con las cartas ya sacadas de la baraja.
 - Crear nuevas cartas.
 - Barajar las cartas.
 - Recorrer la baraja mostrando una por una las cartas que quedan en la baraja.
 - Recorrer la lista de cartas ya sacadas de la baraja mostrándolas una por una.
 - Sacar la primer carta de la baraja.
 - Repartir su mano a cada jugador.
 - Recibir las cartas devueltas, añadiéndolas de nuevo a la baraja y vaciando la lista de cartas sacadas.
- **Cartas:**
 - Asignarle un palo.
 - Asignarle un número.
 - Retornar el palo de la carta.
 - Retornar el número de la carta.
 - Retornar el número y palo como cadena de caracteres para leer sus atributos.
 - **Main:**
 - Inicializar nuevos objetos pertenecientes a las demás clases del proyecto.
 - Emplear los atributos de los objetos creados a partir de las clases del proyecto.

ANÁLISIS SOBRE POSIBLES PATRONES DE DISEÑO APLICABLES:

Ante los requerimientos de funcionamiento del programa, surgen posibles patrones de diseño aplicables al problema. Algunos ejemplos de estos son:

- A la hora de crear una baraja de cartas, una mesa, etc., nos encontramos con el problema de que estos objetos contienen varios otros objetos en ellos. Para esto empleamos listas 'ArrayList', las cuales, al ser una clase preexistente de Java, ya venía, además de con la función de guardar una lista de objetos, con distintos métodos útiles para lo que necesitábamos, como '.add()' para añadir un objeto a la lista o '.remove()' para eliminar un elemento.
- Al recorrer las listas que poseen varias de las clases creadas (Como las listas de Baraja, el mazo de los jugadores, etc.), necesitamos una forma de recorrerlas en su totalidad de manera simple, sin tener que escribir una línea para imprimir cada índice. Para eso empleamos bucles iterativos (En estos casos 'for' y 'for i'), para poder recorrer toda la lista sin tener que escribir de manera independiente la instrucción para cada uno de los índices.
- Al crear un programa donde varios jugadores deben jugar a la vez con una misma baraja, surge la incógnita de cómo podemos llevar esto a cabo. Para ello empleamos el concepto de 'Composición', donde los objetos creados están a su vez compuestos de otros objetos, tal y como una mesa está compuesta por varios jugadores, y los jugadores poseen a su vez un mazo compuesto por varias cartas.

MUESTRA DEL PROGRAMA EN EJECUCIÓN:

```
Run Main x
C:\Users\XP6\.jdk\openjdk-25.0.2\bin\java.exe "-javaagent:C:\Users\XP6\OneDrive\Desktop\Programación\Programación 2\
-> Barajando el mazo...

Cartas en el mazo:
La carta es el 12 de Oro
La carta es el 3 de Espada
La carta es el 2 de Basto
La carta es el 4 de Basto
La carta es el 10 de Espada
La carta es el 12 de Copa
La carta es el 11 de Espada
La carta es el 6 de Basto
La carta es el 4 de Copa
La carta es el 6 de Espada
La carta es el 7 de Oro
La carta es el 5 de Oro
La carta es el 1 de Copa
```

```
Run Main x
-----SACAMOS DOS CARTAS PARA VER EL MÉTODO
La carta es el 12 de Oro
La carta es el 3 de Espada

Cartas en el mazo:
La carta es el 2 de Basto
La carta es el 4 de Basto
La carta es el 10 de Espada
La carta es el 12 de Copa
La carta es el 11 de Espada
La carta es el 6 de Basto
La carta es el 4 de Copa
La carta es el 6 de Espada
La carta es el 7 de Oro
La carta es el 5 de Oro
La carta es el 1 de Copa
La carta es el 5 de Basto
La carta es el 7 de Basto
```

```
Run Main x
-----Repartimos las cartas a todos los jugadores que estén en mesa

Repartiendo cartas a: Leo
La carta es el 2 de Basto
La carta es el 4 de Basto
La carta es el 10 de Espada
La carta es el 12 de Copa

Repartiendo cartas a: Maxi dou
La carta es el 11 de Espada
La carta es el 6 de Basto
La carta es el 4 de Copa
La carta es el 6 de Espada

Repartiendo cartas a: Chadvier
La carta es el 7 de Oro
La carta es el 5 de Oro
La carta es el 1 de Copa
```

```
Run Main x
Repertiendo cartas a: Maxi dou
La carta es el 11 de Espada
La carta es el 6 de Basto
La carta es el 4 de Copa
La carta es el 6 de Espada

Repertiendo cartas a: Chadvier
La carta es el 7 de Oro
La carta es el 5 de Oro
La carta es el 1 de Copa
La carta es el 5 de Basto

Repertiendo cartas a: Lauti God
La carta es el 3 de Basto
La carta es el 1 de Oro
La carta es el 7 de Espada
La carta es el 2 de Espada
```

```
Run Main x
-----CARTAS YA REPARTIDAS

Cartas sacadas:
Ya se sacó el 12 de Oro
Ya se sacó el 3 de Espada
Ya se sacó el 2 de Basto
Ya se sacó el 4 de Basto
Ya se sacó el 10 de Espada
Ya se sacó el 12 de Copa
Ya se sacó el 11 de Espada
Ya se sacó el 6 de Basto
Ya se sacó el 4 de Copa
Ya se sacó el 6 de Espada
Ya se sacó el 7 de Oro
Ya se sacó el 5 de Oro
Ya se sacó el 1 de Copa
Ya se sacó el 5 de Basto
Ya se sacó el 3 de Basto
```

```
Run Main x
-----Las cartas que quedaron son:

Cartas en el mazo:
La carta es el 12 de Basto
La carta es el 6 de Copa
La carta es el 2 de Oro
La carta es el 10 de Oro
La carta es el 4 de Oro
La carta es el 3 de Oro
La carta es el 1 de Espada
La carta es el 5 de Copa
La carta es el 12 de Espada
La carta es el 3 de Copa
La carta es el 4 de Espada
La carta es el 7 de Basto
La carta es el 10 de Copa
La carta es el 11 de Copa
La carta es el 11 de Oro
```